

Relatório - IPPD

Nomes:

Pamela Rosa de Oliveira

Renan de Souza Cazelotto

Nívea Domingues Batoni

Fabio Almeida de Siqueira

Pthreads

Fase de atribuição

Na fase de atribuição, cada ponto é associado ao centróide mais próximo. Para paralelizar essa etapa, dividimos o vetor de pontos entre as threads, de modo que cada uma processa apenas seu próprio intervalo. Como cada thread escreve apenas nos `cluster_id` dos pontos que lhe pertencem e os centróides são apenas lidos, não ocorre a condição de corrida.

Fase de atualização

Já na fase de atualização, a posição de cada centróide é recalculada como a média dos pontos atribuídos ao seu cluster.

Para paralelizar essa etapa, dividimos novamente o conjunto de pontos entre as threads. Cada thread processa apenas a sua parte dos dados, acumulando localmente as somas das coordenadas e o número de pontos pertencentes a cada cluster.

Ao final, esses resultados locais são combinados em estruturas globais por meio de uma etapa de redução. Essa fusão é protegida por um mutex, evitando condições de corrida quando múltiplas threads atualizam os mesmos centróides.

OpenMP

Fase de atribuição.

Em OpenMP, essa etapa é paralelizada usando `#pragma omp parallel for`, que distribui o laço principal entre as threads.

Como cada iteração do laço trata um ponto independente, cada thread lê apenas os centróides e escreve exclusivamente no campo `cluster_id` do seu ponto, evitando qualquer tipo de condição de corrida.

O acesso aos centróides é somente leitura, e cada thread manipula índices distintos do vetor de pontos, garantindo que essa etapa seja naturalmente livre de dependências e totalmente paralelizável.

Fase de atualização

Na fase de atualização, os centróides são recalculados como a média dos pontos associados a cada cluster.

Cada thread recebe um identificador (`tid`) e utiliza vetores locais de somas e contagens, armazenados em regiões separadas do vetor global por thread. Dessa forma, durante a etapa paralela, cada thread acumula de forma independente as somas de coordenadas e as quantidades de pontos de cada cluster.

Após a coleta local, é realizada uma etapa sequencial de redução, na qual as somas e contagens de todas as threads são combinadas para formar os valores globais dos clusters. Como essa fusão é feita fora da região paralela, não há necessidade de mutex.

Por fim, um segundo `#pragma omp parallel for` atualiza as coordenadas dos centróides dividindo as somas acumuladas pela contagem de pontos de cada cluster. Como cada centróide é escrito por apenas uma iteração do laço, essa etapa também não apresenta risco de race condition.

MPI

Fase de atribuição

Na fase de atribuição, cada ponto é associado ao centroide mais próximo. Em MPI, essa etapa começa com a distribuição do conjunto total de pontos entre os processos, de forma que cada processo receba apenas uma fatia dos dados. Os centroides atuais são enviados a todos os processos através de um *MPI_Bcast*, o que garante que todos utilizem as mesmas posições durante a iteração.

Cada processo calcula regionalmente as distâncias entre os pontos que lhe pertencem, determinando o *cluster* mais próximo. Uma vez que cada processo opera exclusivamente sobre sua própria região, a etapa de atribuição se faz, por natureza, livre das Condições de Corrida.

Durante o desenvolvimento, percebeu-se que parte dos broadcasts era desnecessária: todos os processos participam igualmente da atualização dos centróides, o rank 0 não realiza nenhuma operação exclusiva que exija envio posterior e o fluxo reduce-broadcast introduzia uma sobrecarga e mais uma sincronização.

Por isso, essa sequência foi substituída por um único *MPI_Allreduce*, que combina automaticamente os somatórios parciais de todos os processos, retorna o resultado combinado para todos os processos simultaneamente, elimina o broadcast e simplifica a etapa de comunicação, garante que todos os processos avancem imediatamente já com os centroids atualizados. O broadcast inicial (apenas para distribuir os centroides escolhidos em rank 0) foi mantido.

Fase de atualização

Na fase de atualização, os centroides têm de ser recalculados a partir da média dos pontos atribuídos ao seu *cluster*. Paralelizar esta etapa com MPI requer que cada processo acumule localmente as somas das coordenadas dos pontos que pertencem ao seu cluster e a quantidade de pontos que cada cluster recebeu em sua partição. Após o *MPI_Allreduce*, cada processo divide a soma total pela quantidade total de pontos de cada cluster e atualiza as coordenadas dos centróides de forma idêntica e independente. Essa modificação reduz a complexidade da comunicação, encurta o caminho crítico da iteração e melhora a eficiência geral do algoritmo distribuído.

Análise de Desempenho e Discussão

Speedup médio e tempo de execução comparado ao baseline

```
--- Resumo dos Resultados (Checksum de Referência: 5002634) ---
Versão      | Tempo Médio (s) | Speedup | Corretude
-----
Sequencial  | 46.0818         | 1.00x   | (30/30)
OpenMP      | 16.5801         | 2.78x   | (30/30)
Pthreads    | 18.6209         | 2.47x   | (30/30)
MPI         | 23.2004         | 1.99x   | (30/30)
```

```
--- Resumo dos Resultados (Checksum de Referência: 5002634) ---
Versão      | Tempo Médio (s) | Speedup | Corretude
-----
Sequencial  | 46.0818         | 1.00x   | (30/30)
OpenMP      | 16.5801         | 2.78x   | (30/30)
Pthreads    | 18.6209         | 2.47x   | (30/30)
MPI         | 23.2004         | 1.99x   | (30/30)
```

*nota: o *avaliador.py* tentava rodar com mais processos do que o sistema permitia para o mpi, então ele acusava erro, pois nem entrava na execução. Para resolver isso, limitamos o avaliador para funcionar apenas com 4 processos, a alteração foi commitada para registro.

Discussão do overhead e escalabilidade

O overhead da versão com Pthreads surge principalmente da sincronização explícita realizada por meio de mutexes durante a etapa de redução. Embora exista custo de troca de contexto entre as threads, ele é relativamente pequeno comparado à contenção causada quando múltiplas threads tentam atualizar estruturas compartilhadas. A escalabilidade é boa enquanto o número de threads não ultrapassa o número de núcleos físicos; a partir desse ponto, o aumento de contenção e o custo adicional de agendamento reduzem o speedup e tornam o ganho marginal.

Na tabela a seguir, obtida testando a execução da paralelização com OpenMP com diferentes números de threads, podemos observar como o speedup cresce

conforme o número de threads aumenta, até o número máximo de núcleos do processador da máquina em que o programa foi executado (que é 8 nesse caso):

Número de threads	Tempo de execução (em segundos)
1	36.750045
2	21.731084
3	18.281630
4	15.324127
5	13.980532
6	12.671488
7	11.980505
8	11.284151
9	12.671436
10	11.842763
11	11.554323
12	11.250818
13	11.542699
14	12.842704
15	15.100941

Obs: a corretude de todos os testes foi satisfatória (correta), e algumas instabilidades da máquina em que o programa foi executado podem alterar levemente a percepção dos resultados.

Em OpenMP, o comportamento é semelhante ao de Pthreads, mas com um overhead adicional associado ao gerenciamento automático das regiões paralelas e ao particionamento do trabalho pelo runtime. Além disso, barreiras implícitas ao final de loops paralelos introduzem sincronização extra. Assim como em Pthreads, a escalabilidade é limitada pelo número de núcleos disponíveis: acima disso, o custo de sincronização e o aumento da pressão no sistema de memória reduzem a eficiência.

Entre os três modelos, MPI apresenta o maior overhead, principalmente devido ao custo das operações de comunicação entre processos, que não compartilham

memória. A cada iteração, é necessário realizar trocas de mensagens e reduções globais, cujo custo cresce com o número de processos e com a latência da rede. Para datasets pequenos, o tempo gasto em comunicação pode superar o tempo de computação. Por outro lado, para conjuntos de dados maiores, MPI pode escalar melhor, pois cada processo armazena apenas uma fração dos dados, reduzindo o uso de memória local e distribuindo o trabalho de forma mais equilibrada. A escalabilidade, portanto, é fortemente influenciada pela frequência e pelo volume de comunicação realizado ao longo do algoritmo.

Impacto da comunicação e sincronização (especialmente no MPI)

No MPI, o custo de comunicação entre processos têm impacto direto no desempenho, especialmente porque a memória não é compartilhada e cada processo trabalha com uma partição independente dos dados.

Durante a fase de atribuição, o impacto é baixo, pois cada processo opera apenas sobre seus próprios pontos sem necessidade de sincronização intermediária. Entretanto, na fase de atualização, a comunicação torna-se essencial para combinar as informações parciais de cada processo.

Operações como MPI_Allreduce introduzem sincronização global uma vez que todos os processos precisam chegar ao mesmo ponto antes de continuar, o que pode aumentar o tempo total caso exista desequilíbrio de carga ou diferenças de latência entre nós.

Apesar disso, a escolha por MPI_Allreduce (em vez de um Broadcast seguido de agregações manuais) reduz o número de mensagens e elimina trocas desnecessárias, resultando em uma sincronização mais eficiente e menor overhead de comunicação.

Assim, o desempenho final depende tanto da distribuição equilibrada dos dados quanto da minimização das operações coletivas, que tendem a ser os gargalos naturais em algoritmos paralelos distribuídos como o K-means.

Diferentemente do que ocorre em MPI, em Pthreads e OpenMP a comunicação é interna, pois as threads compartilham memória. Isso torna o modelo de programação mais simples, porém exige atenção ao acesso concorrente de dados compartilhados.

No uso de Pthreads, temos controle explícito sobre como os dados são distribuídos e atualizados pelas threads, sendo necessário utilizar mecanismos de sincronização, como mutexes e barreiras, para evitar condições de corrida, como podemos observar na fase de atualização dos centróides.

Já no OpenMP, parte dessa sincronização é feita automaticamente pelo compilador e pelo runtime, o que facilita a implementação, mas pode introduzir um overhead adicional no gerenciamento das threads.